Lab 3: Framing at the Data Link Layer

In this lab, we implement the framing methods that indicate where one frame ends and the next begins. We need to build three things: a sender that wraps a payload into a frame, a transmission medium that carries the frame (and may corrupt it in transit), and a receiver that recovers the original payload. We cover three framing methods (byte count, byte stuffing, and bit stuffing), then measure how much overhead each one adds.

The sender and receiver never touch each other directly. Everything the sender produces passes through the transmission medium first, exactly as a real link works: bytes leave the sender, cross the medium, and only then reach the receiver. Corruption, when it happens, happens inside the medium, after the sender and before the receiver.

Do it in C++ or Python, your choice. Both will be graded against the same published test vectors, so a correct solution in either language produces identical output, byte for byte.

Following has to be achieved:

• implement byte-count framing, then let the medium corrupt one count byte and show that every following frame is lost;
• implement byte stuffing with FLAG and ESC bytes, and recover payloads that themselves contain those bytes
• implement bit stuffing on a real bit stream, packing and unpacking the bits yourself
• Measure and compare the overhead of each method for random data and for worst-case data.

Setup

Use any C++ compiler (`g++ -std=c++17 lab3.cpp -o lab3`) or Python 3 (`python3 lab3.py`). No external bit or serialization libraries: do the bit manipulation yourself. In particular, do not use `bitarray`, `bitstring`, `struct`, `std::bitset`, or any equivalent that performs the packing for you.

Conventions and constants
• `FLAG = 0x7E` (bit pattern 01111110).
• `ESC  = 0x7D`.
• Payloads and frames are sequences of bytes with values 0 to 255. Represent them however your language prefers (`bytes`/`bytearray` in Python, `std::vector<uint8_t>` in C++). Only the produced byte values are graded, not the type.
• Process every byte most-significant-bit first: bit 7 down to bit 0.
• A byte-count field is one byte, so payloads in Part A are at most 255 bytes.

The transmission medium

Every frame the sender produces is handed to the transmission medium, which delivers it to the receiver. Implement the medium as a single operation:
`transmit(frame_bytes, corruptions) -> received_bytes`

• `corruptions` is a list of `(index, value)` pairs. For each pair, the medium overwrites the byte at that position with that value while the frame is in transit.
• With an empty list, the medium delivers the frame unchanged. This is the normal case, used for every round-trip test.
• A corruption represents an error the channel introduces after the sender and before the receiver. A real channel flips individual bits at some rate (the bit-error rate you computed in Lab 2); here the error is specified as an exact resulting byte so that the outcome is fixed and reproducible for grading.

• The receiver only ever sees the output of `transmit`. It must work from the delivered bytes alone, with no knowledge of what the sender originally sent.

Test vectors

All values are in hexadecimal. These are the grading anchor: run your code on each input and confirm the output matches, byte for byte, before submitting.

Method #	**Method Name**	**Input**	**In transit (medium)**	**Expected result**
2	Byte count	Clean delivery	`transmit(..., [])`	Receiver recovers `11 22 33` and `AA` (both frames)
3	Byte count	Corrupted in transit	`transmit(..., [(0, 0x05)])`	Medium delivers `05 11 22 33 01 AA`; receiver recovers one payload `11 22 33 01 AA`; second frame lost (0 of 2 correct)
4	Byte stuffing	`11 22`	clean	`7E 11 22 7E`

Method #	Method Name	Input	In transit (medium)	Expected result
5	Byte stuffing	`7E`	clean	`7E 7D 7E 7E`
6	Byte stuffing	`7D`	clean	`7E 7D 7D 7E`
7	Byte stuffing	`7D 7E`	clean	`7E 7D 7D 7D 7E 7E`
8	Bit stuffing	`FF`	clean	`FB 80`, bit length 9
9	Bit stuffing	`7E`	clean	`7D 00`, bit length 9
10	Bit stuffing	`FF FF`	clean	`FB EF A0`, bit length 19

Part A: Byte-count framing

Implement a framer that prefixes each frame with a one-byte length count and a receiver that reads the count and extracts the payload. Send the frames through the medium: first cleanly, then with the medium corrupting one count byte, and show that the corruption loses every frame after it.

1. Implement `frame(payload)`: return one count byte (the payload length) followed by the payload bytes.
2. Implement `parse(stream)`: given several frames concatenated, return the list of payloads by reading each count and then that many bytes.

3. Clean delivery. Build a stream from the payloads `11 22 33` and `AA`, confirm it equals `03 11 22 33 01 AA`, pass it through `transmit` with no corruption, and confirm `parse` recovers both payloads.
4. Corrupted delivery. Pass the same stream through `transmit` with the corruption `(0, 0x05)`, so the medium delivers `05 11 22 33 01 AA`. Run `parse` on the delivered bytes. Report how many of the two frames are recovered correctly, and explain why the receiver cannot find the boundary of any following frame once the count is wrong.

Part B: Byte stuffing

Implement framing with FLAG delimiters and ESC escaping, send frames through the medium, and recover payloads that contain FLAG and ESC bytes.

Background: each frame begins and ends with a FLAG byte (`0x7E`). If a FLAG byte appears inside the payload it would be mistaken for a boundary, so the sender escapes it: before any FLAG or ESC byte in the payload it inserts an ESC byte (`0x7D`). On the receiving side, a FLAG that is not preceded by ESC is a real boundary, and when the receiver sees an ESC it discards the ESC and takes the following byte literally.

1. Implement `frame(payload)`: output FLAG, then the payload with every FLAG or ESC byte preceded by an ESC, then FLAG.
2. Implement `deframe(frame)`: remove the delimiters, remove each ESC, and return the original payload.
3. Verify round-trip through a clean medium on ordinary data (for example `11 22`): `deframe(transmit(frame(payload), []))` must return the original payload.
4. Test the awkward cases and confirm exact recovery: a payload containing FLAG (`7E`), a payload containing ESC (`7D`), and a payload containing ESC followed by FLAG (`7D 7E`). Match the outputs in the test-vector table.

Part C: Bit stuffing

Implement bit stuffing on a real bit stream. Extract the bits from the payload bytes, apply the stuffing rule, and pack the result back into bytes yourself.

Background: at the bit level the frame delimiter is the pattern 01111110. To stop that pattern from appearing inside the payload, the sender inserts a 0 bit after any run of five consecutive 1 bits. The receiver removes a 0 that follows five consecutive 1 bits. Because stuffing changes the number of bits, the stuffed stream is usually not a whole number of bytes, so you must also track the exact bit length.

1. Implement `bit_stuff(payload)`: read each byte most-significant-bit first, insert a 0 after every five consecutive 1 bits, pack the stuffed bits back into bytes (pad the final byte with 0 bits), and return both the packed bytes and the total number of stuffed bits.
2. Implement `bit_unstuff(stuffed_bytes, bit_length)`: reverse the process and return the original payload bytes.
3. Verify round-trip.

4. Test long runs of 1 bits using payloads `FF` and `FF FF`, and confirm the packed output and bit length match the test-vector table (`FF` gives `FB 80` at bit length 9; `FF FF` gives `FB EF A0` at bit length 19).

Part D: Overhead analysis

Measure the extra bytes or bits each method adds, for random data and for worst-case data, and compare the two methods.
Background: framing adds overhead. Byte stuffing adds one ESC byte for every FLAG or ESC byte in the payload. Bit stuffing adds one bit for every five consecutive 1 bits. The amount therefore depends on the data, and each method has a worst case that you can construct deliberately.

1. For byte stuffing, measure the overhead (framed size minus payload size, as a percentage of payload size) on random payloads and on the worst-case payload. Construct the worst case yourself.
2. For bit stuffing, measure the overhead (stuffed bits minus payload bits, as a percentage) on random payloads and on the worst-case payload. Construct the worst case yourself.
3. State each worst-case input and explain in one line why it is the worst.
4. Compare the two methods' worst-case overhead and state which is larger.

What to submit:
• Your source file (C++ or Python) implementing the sender, the transmission medium, and the receiver for every part.
• Program output showing that Parts A, B, and C reproduce the test-vector table exactly, and that each method round-trips through a clean medium.
• Part A: the number of frames recovered after the medium corrupts the count byte, and the explanation.
• Part D: the measured overheads, and the two worst-case inputs with a one-line justification for each.

Appendix: the interface each program must implement

Implement these operations. Names and types are illustrative; only the byte values you produce are graded.
Transmission medium: `transmit(frame_bytes, corruptions) -> received_bytes`, where `corruptions` is a list of `(index, value)` overwrites applied in transit; an empty list delivers the frame unchanged.
- Byte count: `frame(payload) -> bytes` and `parse(stream) -> list of payloads`.
- Byte stuffing: `frame(payload) -> bytes` and `deframe(frame) -> payload`.
Bit stuffing: `bit_stuff(payload) -> (stuffed_bytes, bit_length)` and `bit_unstuff(stuffed_bytes, bit_length) -> payload`.
Overhead: a routine that reports, for a given payload, the framed size and the overhead percentage for byte stuffing and for bit stuffing.Every send in the lab goes sender to medium to receiver: the receiver operates on the output of `transmit`, never on the sender's bytes directly.